

University of Bahrain

College of Information Technology
Department of Computer Science

ITCS347 Design and Analysis of Algorithms

Exercises #4 – *Divide-and-Conquer*

1. Use *Master Theorem* to find the order of growth for solutions of the following recurrences.
 - (a) $T(n) = 2T(n/3) + 1$
 - (b) $T(n) = 5T(n/4) + n + 1$
 - (c) $T(n) = 7T(n/7) + n$
 - (d) $T(n) = 9T(n/3) + n^2 + 2n - 1$
2. Use *MergeSort* algorithm to sort the array
 - (a) $A = \{32, 15, 14, 15, 11, 17, 25, 51\}$.
 - (b) $A = \{12, 25, 17, 19, 51, 32, 45, 18, 22, 37, 15\}$.
3. Repeat Question 2 on *QuickSort* algorithm.
4. Suppose you are give the list E, X, A, M, P, L, E .
 - (a) Sort the list in nondecreasing alphabetical order using *Merge Sort* Algorithm. How many comparisons are there in *Merge* Algorithm?
 - (b) Sort the list in nondecreasing alphabetical order using *Quick Sort* Algorithm. How many comparisons are there in *Partition* Algorithm?
5. Briefly show the differences between the *Merge Sort* and *Quick Sort* algorithms (in terms of principle, complexity etc.).
6. Let $C_{worst}(n)$ be the number of key comparisons of MergeSort in the worst case. Given that $C_{worst}(1) = 0$,
 - (a) use *Back Substitution* to prove that $C_{worst}(n) = n \log_2 n - n + 1$.
 - (b) use *Master Theorem* to prove that $C_{worst}(n) \in \Theta(n \log n)$.
7. Consider the following algorithm that counts the number of zeros in a positive integer n .

```
1: Algorithm ZEROS( $n$ )
2:   if  $n = 0$  then
3:     return 1
4:   else if  $n < 10$  then
5:     return 0
6:   else if  $n \bmod 10 = 0$  then
7:     return ZEROS( $n/10$ )+1
8:   else
9:     return ZEROS( $n/10$ )
```

- (a) Trace the algorithm for $n = 601009$.
- (b) Write a recursive relation $T(n)$ for the algorithm time complexity (what is the basic operation?).

Solution: The basic operation is the division.

$$T(n) = \begin{cases} 0 & \text{for } 0 \leq n < 10 \\ 1 + T(n/10) + 0 & \text{for } n \geq 10 \end{cases}$$

- (c) Find an explicit formula for $T(n)$ and its efficiency.

Solution: Using the Master's Theorem, $a = 1, b = 10$, and $d = 0$. Since $a = b^d$, then $T(n) \in \Theta(n^d \log n) = \Theta(\log n)$.

Using Back Substitution,

$$\begin{aligned} T(n) &= T(n/10) + 1 \\ &= T(n/10^2) + 1 + 1 \\ &= T(n/10^3) + 1 + 1 + 1 \\ &\vdots \\ &= T(n/10^k) + k \end{aligned}$$

$$\text{Until } T(n/10^k) = T(1) \implies 10^k = n \implies k = \log n.$$

$$\begin{aligned} \therefore T(n) &= T(1) + k \\ &= 0 + \log n \\ &= \log n \end{aligned}$$

- (d) Write the algorithm using iterative method and prove that its complexity is $\Theta(\log n)$.
8. Let $A[0..n-1]$ be an array of n integers and x an integer. Derive a divide-and-conquer algorithm (two equal divisions) to find the frequency of x in A , i.e., the number of times x appears in A . What is the time complexity of your algorithm?
9. Design a search algorithm that divides a sorted array into one-third and two-thirds instead of two halves as in *BinarySearch* algorithm. Analyze the time complexity of the algorithm. Is it faster or slower than *BinarySearch* algorithm.

Solution:

```

1: Algorithm ONETHIRDBINARYSEARCH( $A[0..n-1], start, end, K$ )
2:   if  $start > end$  then
3:     return -1
4:    $mid \leftarrow start + \lfloor \frac{end - start}{3} \rfloor$ 
5:   if  $K = A[mid]$  then return  $mid$ 
6:   else if  $K < A[mid]$  then
7:     return ONETHIRDBINARYSEARCH( $A, start, mid - 1, K$ )
8:   else
9:     return ONETHIRDBINARYSEARCH( $A, mid + 1, end, K$ )

```

Let $T(n)$ be the time complexity of the algorithm. If the basic operation is key comparison, then in the worst case, the algorithm searches either in:

- The smaller part of size $n/3$ or
- The larger part of size $2n/3$.

Since we don't know where the key lies, worst-case assumes the larger one:

$$T(n) = T(2n/3) + O(1)$$

Solving $T(n)$ using back substitution, $T(n) = O(\log_{3/2} n)$.

Since $\log_{3/2} n \approx 1.7095 \log_2 n$, then it is about $1.7 \times$ slower than ordinary binary search.

10. Write a divide-and-conquer algorithm that tests if an array $A[0..n-1]$ is sorted in ascending order or not. The steps,

- (a) Divide the array A into two equal halves (*low* and *high* sub-arrays),
 - (b) recursively check if the low sub-array is sorted (first solution),
 - (c) recursively check if the high sub-array is sorted (second solution), and
 - (d) combine the two solutions.
11. Given a sorted array of distinct integers $A[0..n-1]$, you want to find out whether there is an index i for which $A[i] = i$. Give a divide-and-conquer algorithm that runs in time $O(\log n)$.
12. Let $A[0..n-1]$ be an array of size n with *distinct* integers. An inversion of A is a pair (i, j) of array indices with $i < j$ and $A[i] > A[j]$. For example, let A be the below array,

index $\rightarrow$	0	1	2	3	4	5
$A \rightarrow$	1	3	5	2	4	6

The array has three inversions $(1, 3)$, $(2, 3)$ and $(2, 4)$.

- (a) Write a brute-force algorithm to count the number of inversions in an array. Explain the steps of your algorithm and analyze its time complexity.
 - (b) (Hard) Write a divide-and-conquer algorithm to count the number of inversions in an array. Explain the steps of your algorithm and analyze its time complexity.
13. **Finance.** *Compound Interest Accumulation.*

In finance, the future value of an investment compounded at possibly varying rates r_i over n periods is given by:

$$FV = P \times \prod_{i=1}^n (1 + r_i)$$

where P is the principal and r_i is the interest rate in period i . Design a divide-and-conquer algorithm to compute FV by recursively splitting the product. Write pseudocode, derive the recurrence relation, and analyze the time complexity.

14. **Physics** *Total Kinetic Energy of Moving Bodies.*

The total kinetic energy of n particles is expressed as:

$$E = \sum_{i=1}^n \frac{1}{2} m_i v_i^2$$

where m_i is the mass and v_i is the velocity of the i^{th} particle. Design a divide-and-conquer algorithm that computes the total kinetic energy. Write pseudocode, derive the recurrence relation, and analyze the time complexity.